

Projeto 4

CAMINHOS ENTRE CIDADES MARCIANAS

Disciplina: TI221 – Estrutura de Dados
2o semestre de 2026
Prof. Francisco da Fonseca Rodrigues

Alunos:
Livia Pavin Ribelato - 25146
Theo Yuji Fabiano Naliato Yama - 25155

1. INTRODUÇÃO

Este relatório descreve o desenvolvimento do Projeto IV da disciplina de Estruturas de Dados do 2º ano do curso de Informática do Colégio Técnico de Campinas (Cotuca). O projeto consiste em um aplicativo em C#, com interface Windows Forms, que auxilia os habitantes de Marte a consultar os caminhos possíveis entre as cidades ligadas pela malha aeroviária de dirigíveis da Federação Marciana.

O aplicativo deve encontrar todos os caminhos existentes entre uma cidade de origem e uma cidade de destino escolhidas pelo usuário, utilizando backtracking com pilhas ou backtracking com recursão, e indicar o caminho mais vantajoso de acordo com o critério escolhido: menor distância, menor tempo ou menor custo.

2. DESENVOLVIMENTO

2.1 Arquivos de dados

O programa utiliza dois arquivos de texto com campos separados por ponto e vírgula. O arquivo CidadesMarte.txt contém 23 cidades, cada uma com identificador, nome e coordenadas X e Y proporcionais às dimensões do mapa de Marte. O arquivo CaminhosEntreCidadesMarte.txt contém 28 ligações diretas, cada uma com os identificadores das cidades de origem e de destino, a distância, o tempo de percurso e o custo da passagem. As ligações são bidirecionais.

2.2 Estruturas de dados

Foram utilizadas as seguintes estruturas:

- a) classe Cidade, que guarda identificador, nome e coordenadas, e é ordenada pelo identificador;
- b) classe Ligacao, que guarda os identificadores das duas cidades ligadas, a distância, o tempo e o custo, e fornece o valor do critério escolhido por meio do enumerado CriteriosSeparacao;
- c) vetor de 25 posições de objetos Cidade, mantido em ordem de identificador e consultado por pesquisa binária;
- d) matriz de adjacências de 25 por 25 posições de objetos Ligacao, na qual linha e coluna correspondem aos índices das cidades no vetor, e cada ligação é armazenada nas posições simétricas por ser bidirecional;

e) classe genérica `PilhaLista`, implementada com nós encadeados (`NoLista`), que oferece as operações `Empilhar`, `Desempilhar`, `OTopo`, `Conteudo` e `ConteudoInvertido`.

2.3 Leitura dos arquivos

Ao carregar o formulário `FrmCaminhos`, o vetor e a matriz são criados e o programa solicita ao usuário, por meio de um `OpenFileDialog`, o arquivo de cidades e, em seguida, o arquivo de ligações. Cada linha é lida pelo método `LerRegistro` da respectiva classe, que separa os campos pelo caractere “;”. As cidades são inseridas no vetor em ordem de identificador. Para cada ligação, a pesquisa binária determina os índices das duas cidades, que passam a ser a linha e a coluna da matriz em que o objeto é guardado. Concluída a leitura, os objetos do vetor são carregados nos comboboxes `cbOrigem` e `cbDestino`, com a propriedade `DisplayMember` configurada para exibir o nome da cidade.

2.4 Busca de caminhos com pilha

O método `ProcuraCaminhoPilha` percorre a matriz de forma iterativa. A partir da cidade de origem, examina a linha da matriz em busca de uma ligação existente para uma cidade ainda não visitada. Quando encontra, empilha a ligação, marca a cidade como visitada e passa a examinar a linha da cidade seguinte. Quando a cidade alcançada é o destino, a pilha é copiada para a lista de caminhos encontrados e o último trecho é desempilhado, permitindo continuar a busca. Quando não há mais opções em uma cidade, ocorre o `backtracking`: a cidade é desmarcada, a última ligação é desempilhada e a busca retorna à cidade anterior, prosseguindo pela coluna seguinte. O processo termina quando todas as possibilidades a partir da origem foram esgotadas.

2.5 Busca de caminhos com recursão

O método `ProcuraCaminhoRecursivo` recebe a cidade atual, a lista de ligações do caminho em construção e o vetor de cidades visitadas. Se a cidade atual é o destino, uma cópia do caminho é adicionada à lista de caminhos encontrados. Caso contrário, o método percorre a linha da matriz e, para cada cidade vizinha não visitada, adiciona a ligação ao caminho, marca a cidade, chama a si mesmo e, ao retornar, remove a ligação e

desmarca a cidade. O retorno da chamada recursiva realiza o backtracking dispensando a pilha explícita.

2.6 Interface do programa

O formulário contém os seguintes controles:

- a) comboboxes cbOrigem e cbDestino, para escolha das cidades;
- b) opções “Pilhas” e “Recursão”, para escolha da técnica de busca;
- c) opções “Distância”, “Tempo” e “Preço”, para escolha do critério de peso;
- d) botão “Achar caminho”, que dispara a busca;
- e) DataGridView dgvCaminhos, que exibe cada caminho encontrado em uma coluna, com o nome das cidades do trajeto;
- f) DataGridView dgvMelhorCaminho, destinado ao melhor caminho segundo o critério escolhido;
- g) painel pnlMapa, que exibe o mapa de Marte ajustado ao tamanho do painel.

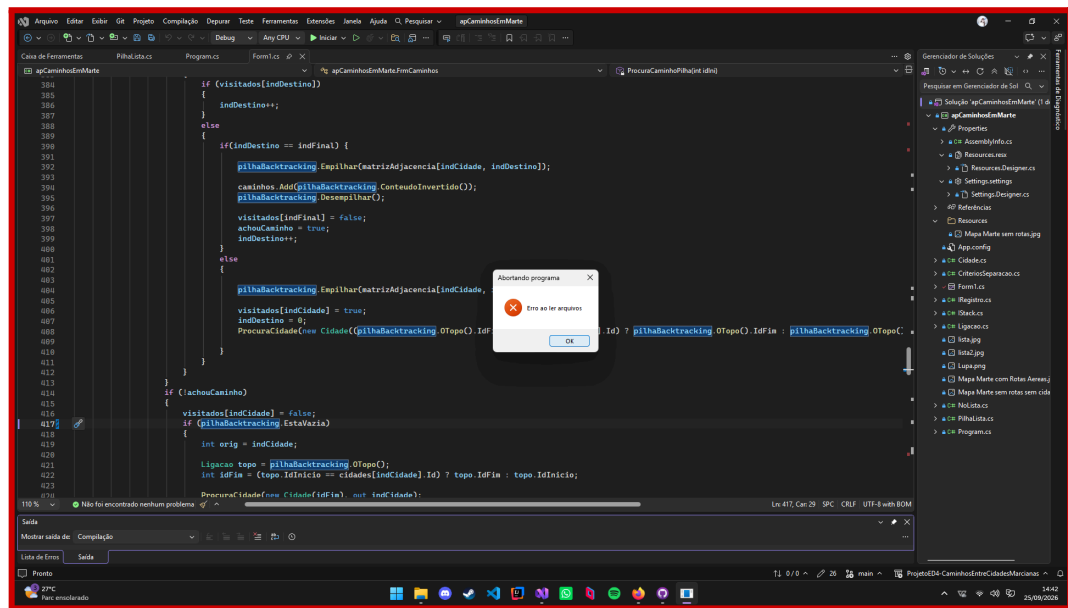
2.7 Execução do programa

ERROS ENCONTRADOS

Durante o desenvolvimento, foram encontrados os quatro erros descritos a seguir.

3.1 Erro ao abrir arquivo

Ocorreu nos métodos Form1_Load, LeuArquivoCidades, LeuArquivoLigacoes e LerRegistro. Havia falha na abertura e na leitura dos arquivos de cidades e de ligações: quando a caixa de diálogo era cancelada, o programa exibia a mensagem “Erro ao ler arquivos” e era encerrado, e linhas fora do formato esperado (quantidade de campos diferente do previsto ou valores numéricos inválidos) impediam a leitura correta dos registros.



3.2 Erro ao fazer backtracking de pilha

Ocorreu no método ProcuraCaminhoPilha. Havia dificuldade em controlar o retorno à cidade anterior: ao desempilhar uma ligação, era preciso recuperar a cidade de origem, retomar a varredura da linha da matriz a partir da coluna seguinte e desmarcar a cidade no vetor de visitados. Além disso, como a mesma ligação é guardada nas duas posições simétricas da matriz, os campos IdInicio e IdFim nem sempre correspondiam ao sentido em que o trecho estava sendo percorrido.

3.3 Erro ao inverter o conteúdo da pilha

Ocorreu no método ConteudoInvertido. A pilha guarda os trechos do último para o primeiro, sendo necessário invertê-la para obter o caminho na ordem do percurso. O método desempilhava todos os elementos da pilha original para uma pilha auxiliar, o que esvaziava a pilha de backtracking e provocava a exceção “Underflow da pilha” na chamada seguinte de Desempilhar.

3.4 Erro ao exibir caminhos

Ocorreu no método ExibirCaminhos. Cada coluna do dgvCaminhos recebia apenas o nome da cidade de início de cada ligação, de modo que a cidade final do caminho não era listada e, pela bidirecionalidade das ligações, o nome exibido nem sempre correspondia ao sentido do percurso. Além

disso, o índice do melhor caminho não era atualizado durante a comparação dos parâmetros.

4 CONSIDERAÇÕES FINAIS

O projeto permitiu aplicar os conceitos de vetor ordenado com pesquisa binária, matriz de adjacências, pilha encadeada, backtracking iterativo e recursivo. Estão implementadas a leitura dos arquivos, o armazenamento em vetor e matriz, o preenchimento dos comboboxes, as duas técnicas de busca e a listagem dos caminhos encontrados no dgvCaminhos.

Permanecem pendentes o desenho do mapa com as cidades e as rotas (o método pnlMapa_Paint ainda não foi programado), o traçado do trajeto ao selecionar uma linha dos DataGridView e a exibição do melhor caminho no dgvMelhorCaminho.